

Part 1

Task 1.1

Relation A: Employee

Employee(EmplID, SSN, Email, Phone, Name, Department, Salary)

1. list at least 6 different superkeys

- A. EmplID, SSN
- B. SSN
- C. EmplID, Email
- D. Phone, Name, SSN
- E. Department, Salary, EmplID
- F. EmplID

2. Identify all candidate keys

- A. EmplID
- B. SSN
- C. Email
- D. Phone

3. Which candidate key would you choose as primary key and why?

I would choose EmplID because it is short and easy to remember and it cannot be null and it is unique.

4. Can two employees have the same phone number? Justify your answer based on the data shown.

It seems that the phone number of each employee ends with their EmplID, so it should be unique.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Determine the minimum attributes needed for the primary key

5 attributes, StudentID, CourseCode, Section, Semester, Year.

2. Explain why each attribute in your primary key is necessary

StudentID is needed to distinguish which student registered for which things, because there may be 2 students that picked the same thing

CourseCode is needed because a student can enroll in multiple distinct courses within the same semester and year.

Section is needed because a student could theoretically take different sections or components of a course

Semester is needed because of rule 1 which states that a student can take the same course in different semesters, without this attribute a student could pick the exact same thing but in different semester and we would get a duplicate

Year is needed to distinguish between same semesters (fall, spring) across years (fall 2026 and fall 2027 are different because of the year)

3. Identify any additional candidate keys (if they exist)

There are no additional candidate keys because remaining attributes Grade and Credits do not hold any additional information that helps in identifying unique rows

Task 1.2

Given Tables:

Student(StudentID, Name, Email, Major, AdvisorID)
Professor(ProfID, Name, Department, Salary)
Course(CourseID, Title, Credits, DepartmentCode)
Department(DeptCode, DeptName, Budget, ChairID)
Enrollment(StudentID, CourseID, Semester, Grade)

1. Identify all foreign key relationships

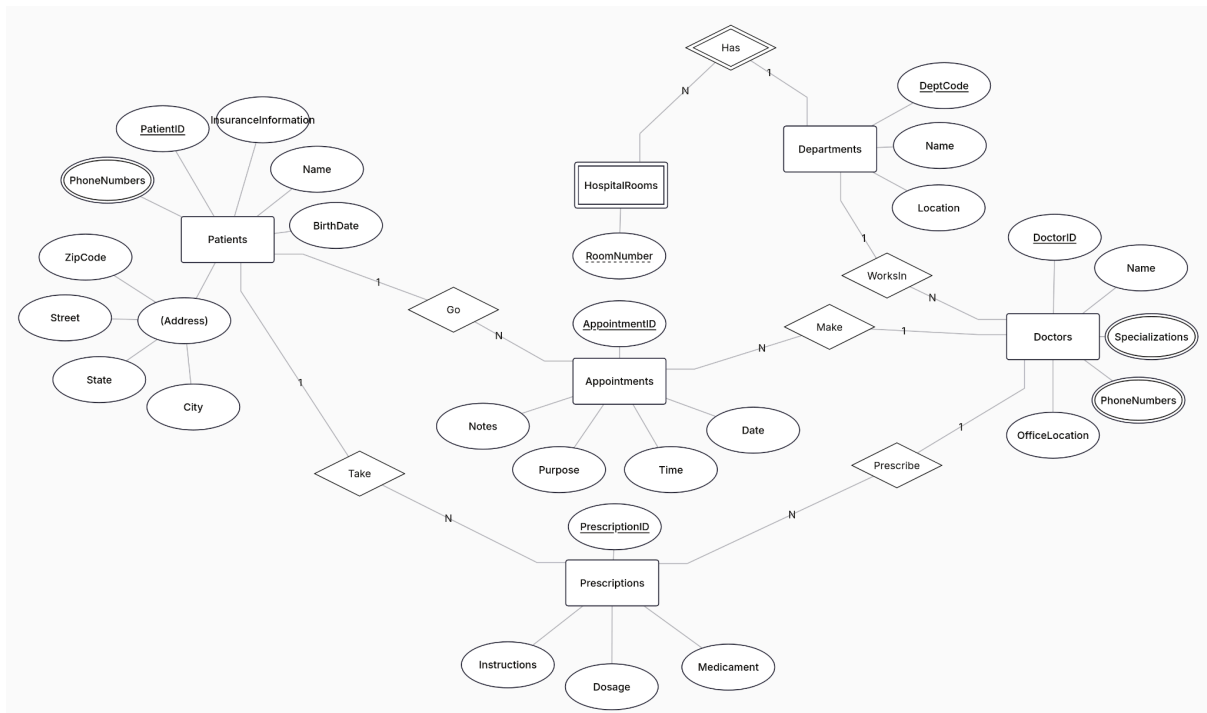
The Enrollment table uses StudentID from the Student table and CourseID from the Course table as foreign keys. The Course table uses DepartmentCode from the Department table as a foreign key. The Professor table uses the Department which I assume is DeptCode from the Department table as a foreign key. The Student table uses AdvisorID which will be a foreign key when the Advisor table will be made.

Part 2

Task 2.1

(ER diagram on the next page, also available via this [link](#))

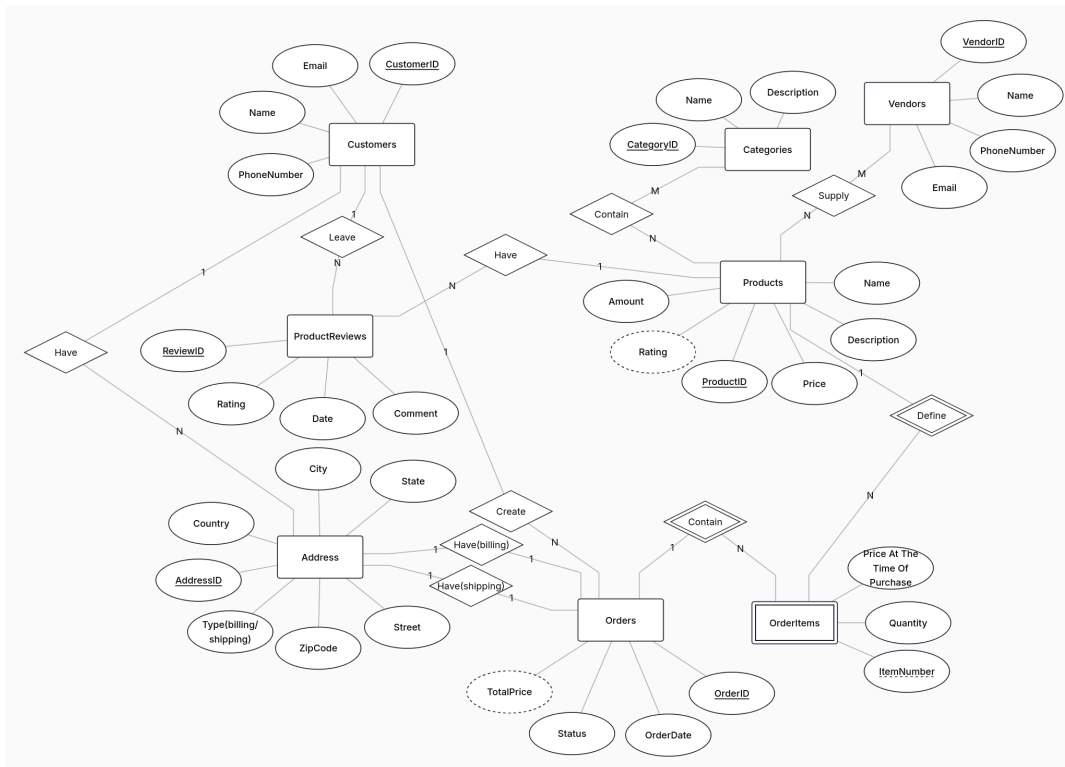
https://github.com/DaZZeNDaZZeN/kbtu_db/blob/main/Lab%201/Task2_1.png



Task 2.2

(ER diagram is also available via this [link](https://github.com/DaZZeNDaZZeN/kbtu_db/blob/main/Lab%201/Task2_2.png))

https://github.com/DaZZeNDaZZeN/kbtu_db/blob/main/Lab%201/Task2_2.png



Part 4

Task 4.1

Given Table:

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. List all FDs in the format A -> B

StudentID -> StudentName, StudentMajor

ProjectID -> ProjectTitle, ProjectType, SupervisorID

SupervisorID -> SupervisorName, SupervisorDept

StudentID, ProjectID -> Role, HoursWorked, StartDate, EndDate

2. What redundancy exists in the table? Give specific examples of update, insert, and delete anomalies.

Redundancies:

StudentID defines StudentName and StudentMajor, so StudentName and StudentMajor are redundant, same logic with ProjectTitle, ProjectType, SupervisorName, SupervisorDept. They can be kept in separate tables and referenced by their ID. StudentID, ProjectID and SupervisorID can be stored as foreign keys.

Anomalies.

In this state if we want to enroll a new student we need to assign them a project. If we want to change the supervisor name we need to change it everywhere or data becomes inconsistent. If we want to delete the supervisor then we need to delete the supervisor or a student then we need to delete all associated projects.

3. Apply 1NF.

All attributes seem to be singlevalued so the table is already in 1NF state, assuming "Role" does not mean multiple roles.

4. Apply 2NF

Students(StudentID, StudentName, StudentMajor)

Projects(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)

StudentProject(StudentID(FK), ProjectID(FK), Role, HoursWorked, StartDate, EndDate)

5. Apply 3NF

Students(StudentID, StudentName, StudentMajor)
 Projects(ProjectID, ProjectTitle, ProjectType, SupervisorID(FK))
 Supervisors(SupervisorID, SupervisorName, SupervisorDept)
 StudentProject(StudentID(FK), ProjectID(FK), Role, HoursWorked, StartDate, EndDate)

Task 4.2

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName,
 InstructorID, InstructorName, TimeSlot, Room, Building)

1. Determine the primary key.

Primary key is (StudentID, TimeSlot), it is easy to see with FDs from below.

2. List all FDs.

StudentID -> StudentMajor
 InstructorID -> InstructorName
 Room -> Building
 TimeSlot, Room -> InstructorID, CourseID
 CourseID -> CourseName

Also one student can not be in two rooms at the same time but in our wsp it is possible to have that kind of schedule somehow :)

StudentID, TimeSlot -> Room

InstructorID, TimeSlot -> CourseID, Room (An instructor can only teach one class at a time)

Room, TimeSlot -> CourseID, InstructorID (A specific room at a specific time can only host one course section and one instructor)

3. Check if the table is in BCNF.

It is not in BCNF because StudentID in first FD is not superkey

4. If not in BCNF, decompose it to BCNF.

Student(StudentID(PK), StudentMajor)
 Course(CourseID(PK), CourseName)
 Instructor(InstructorID(PK), InstructorName)
 Location(Room(PK), Building)
 Section(TimeSlot(PK), Room(PK), CourseID, InstructorID)
 Enrollment(StudentID(PK), TimeSlot(PK), Room) (Room serves as a foreign key back to the Section table).

5. Explain any potential loss of information in your decomposition

I think this decomposition is lossless, as it preserves all existing information.

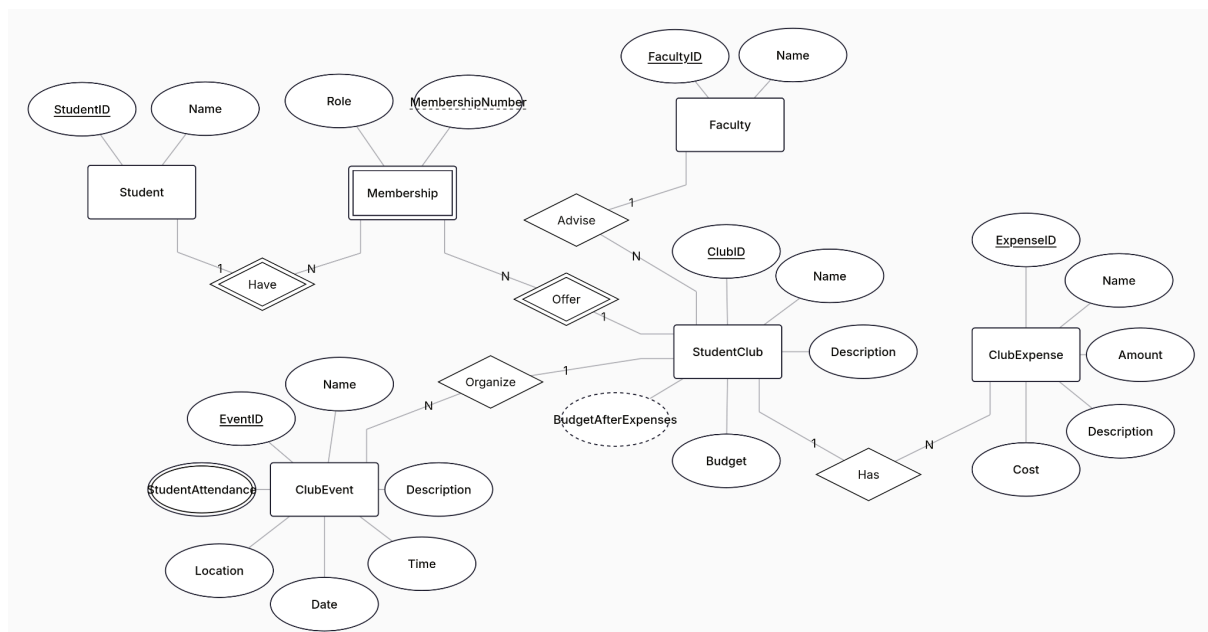
Part 5

Task 5.1

1. Create a complete ER diagram

(ER diagram is also available via this [link](#))

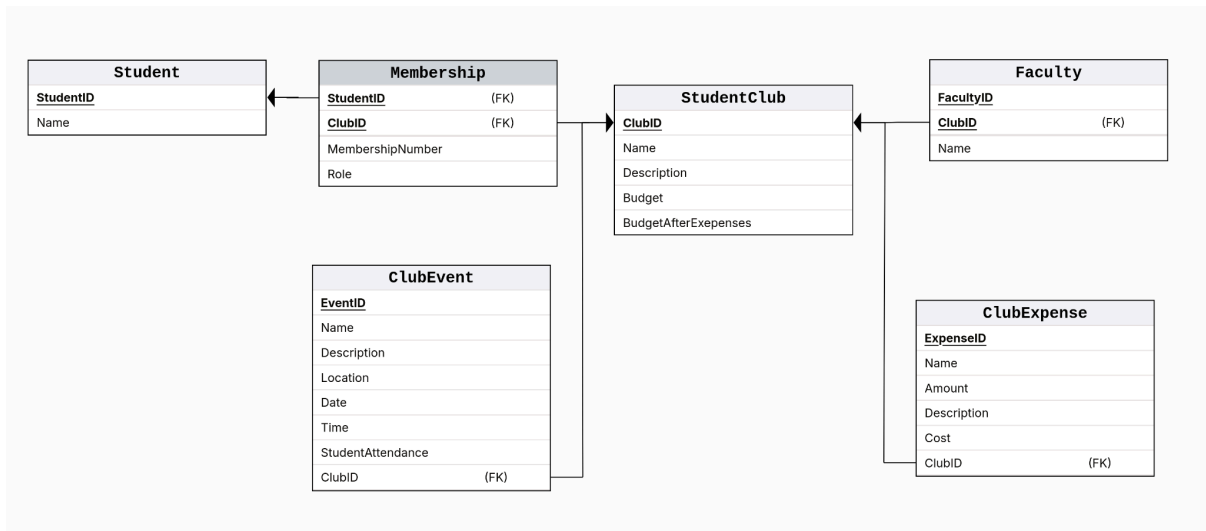
https://github.com/DaZZeNDaZZeN/kbtu_db/blob/main/Lab%201/Task5_1ERD.png



2. Convert your ER diagram to a normalized relational schema

(RS on the next page, also available via this [link](#))

https://github.com/DaZZeNDaZZeN/kbtu_db/blob/main/Lab%201/Task5_1RS.png



3. Identify at least one design decision where you had multiple valid options and explain your choice

I could've made tables for rooms and locations but I decided that I'm too lazy for that and assumed that the club event table is enough for location reservation. Well I'm honest about it.

4. Write 3 example queries that your database should support (in English, not SQL)

1. Get all students with role president in membership
2. Get all events at some location
3. Get all expenses of a certain club which are greater than 100\$
4. Get all events that did not happen yet(will happen in the future)
5. Get student club with most budget